Université de Monastir
Institut Supérieur d'Informatique et de Mathématiques
Département Informatique

Chapitre 2.
Programmation avancée en java

Cours Java avancée- MPGL1

Année Universitaire: 2025-2026

Partie 1.
Génériques avancées en java

Bornes (bounds)

1. Borne supérieure : <? extends T>

Signifie : *“un type inconnu qui est T ou sous-classe de T”*.

Utilisation : lecture → tu peux lire des éléments en tant que T, mais pas écrire (sauf null).

Exemple :

```
List<? extends Number> list1 = new ArrayList<Integer>();

Number n = list1.get(0); // java.lang.IndexOutOfBoundsException: Index 0 out of bounds for length 0

 list1.add(3); // Erreur : on ne peut pas ajouter
```

Bornes (bounds)

Erreur vient de **la covariance avec `<? extends T>`** :

- `List<? extends Number>` signifie : “une liste de **type inconnu qui hérite de Number**”.
- Comme le type exact n’est pas connu, **Java interdit d’ajouter quoi que ce soit**, sauf `null`.
- Sinon le compilateur ne peut pas garantir la **sécurité du type**.

Solution: Utiliser borne inférieure `<? Super Number>`

Bornes (bounds)

2. Borne inférieure : <? super T>

Signifie : *“un type inconnu qui est T ou super-classe de T”.*

Utilisation : écriture → tu peux ajouter des T dans la collection, mais la lecture n’est sûre qu’en tant qu’Object.

Exemple :

```
List<? super Integer> list2 = new ArrayList<Number>();

list2.add(10); // OK : on ajoute un Integer

Object obj = list2.get(0); // Retourne un Object

System.out.println( obj.getClass()); // java.lang.integer

System.out.println(list2.size()); //1
```

Exemple d’utilisation: méthode copy

Un exemple classique est d’implémenter une méthode **de copie de collections** :

```
public static <T> void copy(List<? super T> dest, List<? extends T> src) {
    for (T item : src) {
        dest.add(item);
    }
}
```

```
List<Integer> source = Arrays.asList(1, 2, 3);
List<Number> destination = new ArrayList<>();

copy(destination, source);

System.out.println(destination); // [1, 2, 3]
```

Partie 2.
La réflexivité

La réflexion en java

Définition

- La réflexion est une fonctionnalité avancée du langage de programmation Java.
- Elle permet à un programme Java en cours d'exécution d'examiner ou de s'«introspecter » lui-même, et de manipuler ses propriétés internes (classes, méthodes, attributs).
- Par exemple, il est possible pour une classe Java d'obtenir les noms de tous ses membres et de les afficher.

Mise en place pour utiliser la réflexion

Pour travailler avec l'API de réflexion, on suit généralement trois étapes :

- **Étape 1 : Obtenir un objet Class**

Un objet de type java.lang.Class représente une classe ou une interface en cours d'exécution.

Il existe plusieurs façons de l'obtenir :

- Charger une classe par son nom complet

```
Class<?> c1 = Class.forName("java.lang.String");
```

- Accéder directement à une classe connue

```
Class<?> c2 = String.class;
```

- Obtenir la classe d'un type primitif via le wrapper: **Class<?> c3 = Integer.TYPE**; // équivalent à int.class

Mise en place pour utiliser la réflexion

- **Étape 2 : Explorer les membres de la classe**

Une fois l'objet Class obtenu, on peut l'interroger pour récupérer :

- ses méthodes (***getDeclaredMethods***())
- ses champs (***getDeclaredFields***())
- ses constructeurs (***getDeclaredConstructors***())

Exemple: lister toutes les méthodes d'une classe c1 :

```
Method[] methods = c1.getDeclaredMethods();
for (Method m : methods) {
   System.out.println(m);
}
```

Mise en place pour utiliser la réflexion

- **Étape 3 : Manipuler les membres**

Il est possible d'appeler dynamiquement une méthode ou d'accéder à un champ.

Exemple : appel de la méthode length() sur une chaîne de caractères :

```
String str = "Hello Reflection!";
Method lengthMethod = c1.getMethod("length");
int result = (int) lengthMethod.invoke(str);
System.out.println("Longueur = " + result);
```

Mise en place pour utiliser la réflexion

Exemple de code:

```
Class c = Class.forName("java.lang.String");
Method m[] = c.getDeclaredMethods();
System.out.println(m[0].toString());
```

=> affichera une représentation textuelle de la première méthode déclarée dans String.

Code Complet : Manipulation de Personne avec Réflexion

Soit la classe Personne suivante:

```
class Personne {
    private String nom;
    private int age;
    // Constructeur par défaut
    public Personne() {
        this.nom = "Inconnu";
        this.age = 0;
    }
    // Constructeur avec paramètres
    public Personne(String nom, int age) {
        this.nom = nom;
        this.age = age;
    }
    public String getNom() {
        return nom;
    }
    public void setNom(String nom) {
        this.nom = nom;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public void afficher() {
        System.out.println("Nom : " + nom + ", Âge : " +
age);
    }
}
```

Code Complet : Manipulation de Personne avec Réflexion

Code TestReflection:

```
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
public class TestReflection {
   public static void main(String[] args) {
      try {
         // 1. Récupérer la classe Personne
         Class<?> clazz = Class.forName("Personne");
```

```
   // 2. Afficher les champs
   System.out.println("---- Champs de la classe ----");
      Field[] fields = clazz.getDeclaredFields();
      for (Field f : fields) {
         System.out.println(f.getName() + " : " +
f.getType());
      }

   // 3. Afficher les méthodes
   System.out.println("\n Méthodes de la classe ");
   Method[] methods =
clazz.getDeclaredMethods();
      for (Method m : methods) {
         System.out.println(m.getName());
      }
```

Code Complet : Manipulation de Personne avec Réflexion

```
// 4. Créer une instance avec le constructeur par défaut
        System.out.println("\n---- Instanciation par défaut ----");
        Object p1 = clazz.getDeclaredConstructor().newInstance();
        Method afficher = clazz.getMethod("afficher");
        afficher.invoke(p1);

// 5. Créer une instance avec constructeur paramétré
        System.out.println("\n---- Instanciation avec paramètres ----");
        Constructor<?> ctor = clazz.getDeclaredConstructor(String.class, int.class);
        Object p2 = ctor.newInstance("Alice", 25);
        afficher.invoke(p2);
```

```
// 6. Modifier un champ privé (nom)
        System.out.println("\n---- Modification champ privé ----");
        Field nomField = clazz.getDeclaredField("nom");
        nomField.setAccessible(true); // rendre accessible le champ privé
        nomField.set(p2, "Bob");
        afficher.invoke(p2);
// 7. Gestion des exceptions de Réflexion
    } catch (ClassNotFoundException e) {
        System.err.println("Classe introuvable : " + e.getMessage());
    } catch (NoSuchMethodException e) {
        System.err.println("Méthode/constructeur introuvable : " + e.getMessage());
    } catch (InstantiationException e) {
        System.err.println("Impossible d'instancier la classe : " + e.getMessage());
    } catch (IllegalAccessException e) {
        System.err.println("Accès interdit : " + e.getMessage());
    } catch (InvocationTargetException e) {
        System.err.println("Erreur lors de l'appel d'une méthode : " + e.getCause());
    } catch (NoSuchFieldException e) {
        System.err.println("Champ introuvable : " + e.getMessage());
    }
  } }
```

Exercice d’application

- **Manipulation de la classe ArrayList**

Écrire un programme qui :

1. Charge la classe java.util.ArrayList via Class.forName().
2. Affiche tous les constructeurs de la classe.
3. Crée dynamiquement une instance de ArrayList.
4. Ajoute des éléments en utilisant la méthode add() invoquée par réflexion.
5. Affiche le contenu de la liste.